Addressing underfitting and overfitting through validation curve

Procedure:

- Load the Breast Cancer Wisconsin dataset using Pandas.
- Separate the input features `X` and target variable `y`.
- Encode the target labels using `LabelEncoder`.
- Split the dataset into **80% training data and 20% testing data**.
- Standardize the features using `StandardScaler`.
- Create a **Logistic Regression** model using a pipeline.
- Generate the **learning curve** to study training and validation accuracy.
- Plot the **high bias, high variance, and good bias-variance trade-off** curves.
- Generate the **validation curve** by varying the parameter `C`.
- Plot the training and validation accuracy to diagnose **overfitting and underfitting**.

Solution:

```
import pandas as pd

df = pd.read_csv(r'C:\Users\Lenovo\Downloads\wdbc.csv', header=None)

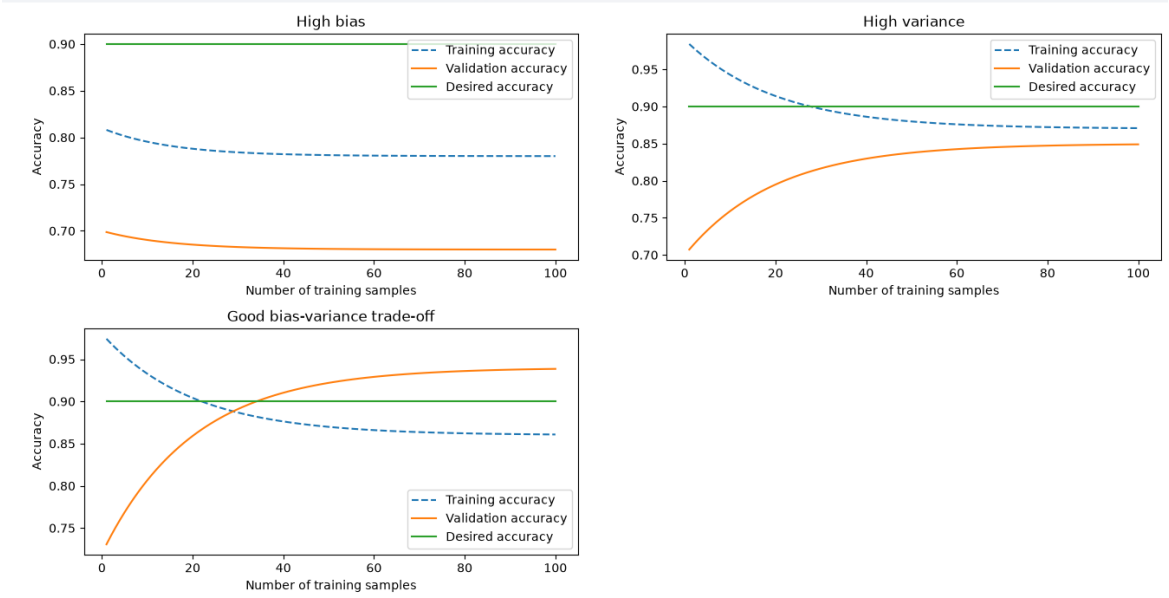
#df.info()
df.head()

from sklearn.preprocessing import LabelEncoder
X = df.loc[:, 2:].values
y = df.loc[:, 1].values
le = LabelEncoder()
y = le.fit_transform(y)

le.transform(['M', 'B'])

from sklearn.model_selection import train_test_split
```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)




```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe_lr = Pipeline([('scl', StandardScaler()),
                    ('clf', LogisticRegression(penalty='l2', random_state=0))])

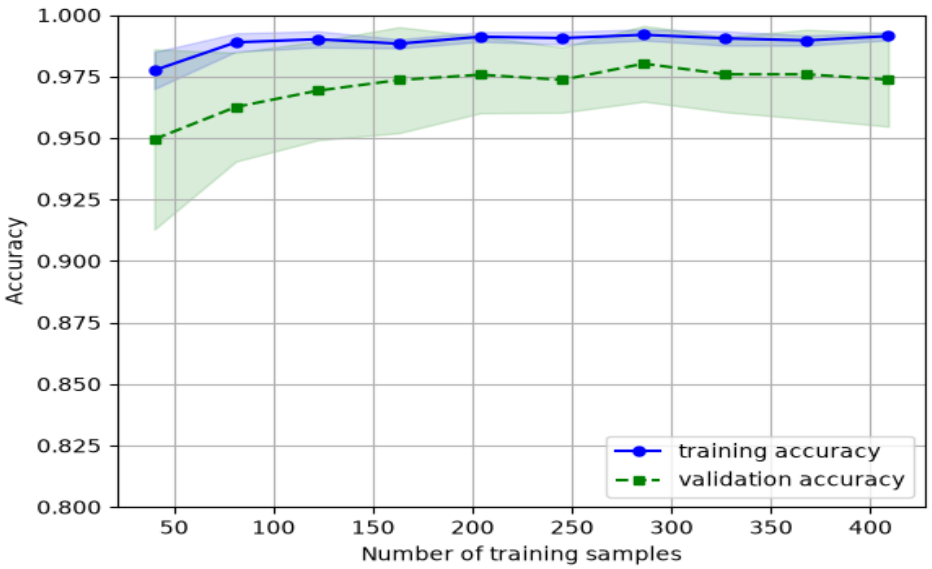
train_sizes, train_scores, test_scores = learning_curve(estimator=pipe_lr,
X=X_train, y=y_train,train_sizes=np.linspace(0.1, 1.0, 10),cv=10,n_jobs=1)

train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

# Train Score
plt.plot(train_sizes, train_mean, color='blue', marker='o', markersize=5, label='training
accuracy')

plt.fill_between(train_sizes,
                 train_mean + train_std,
                 train_mean - train_std,
                 alpha=0.15, color='blue')
```

```
# Test Score
plt.plot(train_sizes, test_mean, color='green', linestyle='--', marker='s', markersize=5,
label='validation accuracy')
plt.fill_between(train_sizes,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')
plt.grid()
plt.xlabel('Number of training samples')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim(0.8, 1.0)
plt.show()
```




```
# Diagnosing bias and variance problems with learning curves

x = np.linspace(1, 100, 100)

# High Bias
training_high_bias = 0.78 + 0.03 * np.exp(-x / 15)
validation_high_bias = 0.68 + 0.02 * np.exp(-x / 15)
desired = np.full(100, 0.90)

# High Variance
training_high_variance = 0.99 - 0.12 * (1 - np.exp(-x / 20))
validation_high_variance = 0.70 + 0.15 * (1 - np.exp(-x / 20))

# Good Bias-Variance Trade-off
```

```
training_good = 0.98 - 0.12 * (1 - np.exp(-x / 20))
validation_good = 0.72 + 0.22 * (1 - np.exp(-x / 20))

fig, axes = plt.subplots(2, 2, figsize=(10, 7))

# High Bias
axes[0, 0].plot(x, training_high_bias, '--', label='Training accuracy')
axes[0, 0].plot(x, validation_high_bias, label='Validation accuracy')
axes[0, 0].plot(x, desired, label='Desired accuracy')
axes[0, 0].set_title('High bias')
axes[0, 0].set_xlabel('Number of training samples')
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].legend()

# High Variance
axes[0, 1].plot(x, training_high_variance, '--', label='Training accuracy')
axes[0, 1].plot(x, validation_high_variance, label='Validation accuracy')
axes[0, 1].plot(x, desired, label='Desired accuracy')
axes[0, 1].set_title('High variance')
axes[0, 1].set_xlabel('Number of training samples')
axes[0, 1].set_ylabel('Accuracy')
axes[0, 1].legend()

# Good Bias-Variance Trade-off
axes[1, 0].plot(x, training_good, '--', label='Training accuracy')
axes[1, 0].plot(x, validation_good, label='Validation accuracy')
axes[1, 0].plot(x, desired, label='Desired accuracy')
axes[1, 0].set_title('Good bias-variance trade-off')
axes[1, 0].set_xlabel('Number of training samples')
axes[1, 0].set_ylabel('Accuracy')
axes[1, 0].legend()

# Remove empty fourth plot
axes[1, 1].axis('off')

plt.tight_layout()
plt.show()


from sklearn.model_selection import validation_curve

param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]

train_scores, test_scores = validation_curve(estimator=pipe_lr,X=X_train,
y=y_train,param_name='clf__C',param_range=param_range,cv=10)
```

```
train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.plot(param_range, train_mean, color='blue', marker='o', markersize=5,
label='training accuracy')

plt.fill_between(param_range,
                 train_mean + train_std,
                 train_mean - train_std,
                 alpha=0.15, color='blue')

plt.plot(param_range, test_mean, color='green', linestyle='--', marker='s',
markersize=5, label='validation accuracy')

plt.fill_between(param_range,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')

plt.grid()
plt.xscale('log')
plt.legend(loc='lower right')
plt.xlabel('Parameter C')
plt.ylabel('Accuracy')
plt.ylim(0.8, 1.0)
plt.show()
```

